

Computación II

Implementación en Python para resolver el problema de las N-Reinas usando backtracking:

```
def imprimir_tablero(tablero, N):
    """Imprime el tablero en formato visual."""
    for fila in tablero:
        linea = ""
        for col in range(N):
            if col == fila:
                linea += " Q "
            else:
                linea += " . "
        print(linea)
    print("\n")

def es_seguro(reinas, fila, col):
    """Verifica si se puede colocar una reina en (fila, col)."""
    for i in range(fila):
        if reinas[i] == col or \
            reinas[i] - i == col - fila or \
            reinas[i] + i == col + fila:
            return False
    return True

def resolver_n_reinas(N):
    """Resuelve el problema de las N-Reinas y devuelve todas las soluciones."""
    soluciones = []
    reinas = [-1] * N

    def backtrack(fila):
        if fila == N:
            soluciones.append(reinas[:])
            return
        for col in range(N):
            if es_seguro(reinas, fila, col):
                reinas[fila] = col
                backtrack(fila + 1)
                reinas[fila] = -1

    backtrack(0)
    return soluciones

if __name__ == "__main__":
    N = int(input("Introduce el valor de N (tamaño del tablero): "))
    soluciones = resolver_n_reinas(N)
    print(f"Se encontraron {len(soluciones)} soluciones para {N}-Reinas.\n")

    for idx, sol in enumerate(soluciones, 1):
        print(f"Solución {idx}:")
        imprimir_tablero(sol, N)
```

Explicación rápida

1. **Representación:** el tablero se representa con una lista `reinas`, donde el índice es la fila y el valor es la columna donde está la reina.
Ejemplo: `reinas = [1, 3, 0, 2]` significa que en fila 0 está en col 1, fila 1 en col 3, etc.
 2. **Seguridad (`es_seguro`):** se revisa que no haya otra reina en la misma columna o en diagonales.
 3. **Backtracking (`backtrack`):** coloca reinas fila por fila, retrocede si no encuentra posición válida.
 4. **Resultados:** imprime todas las soluciones con un tablero visual de `Q` y `.`
-

★ ¿Qué hace este programa?

El **problema de las N-Reinas** es un clásico de la programación y las matemáticas.

Se trata de **colocar N reinas en un tablero de ajedrez de NxN casillas**, de modo que **ninguna pueda atacar a otra**.

En ajedrez, una **reina** puede moverse:

- En **horizontal** (toda la fila).
- En **vertical** (toda la columna).
- En **diagonal** (ambas direcciones).

Por eso, el reto es encontrar posiciones donde ninguna reina pueda "comerse" a otra.

🔗 ¿Por qué usamos `Q` y `.`?

- "`Q`" representa a una **reina colocada** en esa casilla.
- "`.`" representa una **casilla vacía**.

Es solo una manera textual de "dibujar" el tablero en la consola.

Piensa que "`.`" son los espacios libres del tablero y "`Q`" marca dónde está la reina.

♔ Ejemplo con 4-Reinas

Supongamos que el programa encuentra esta solución para un tablero de 4x4:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Esto significa:

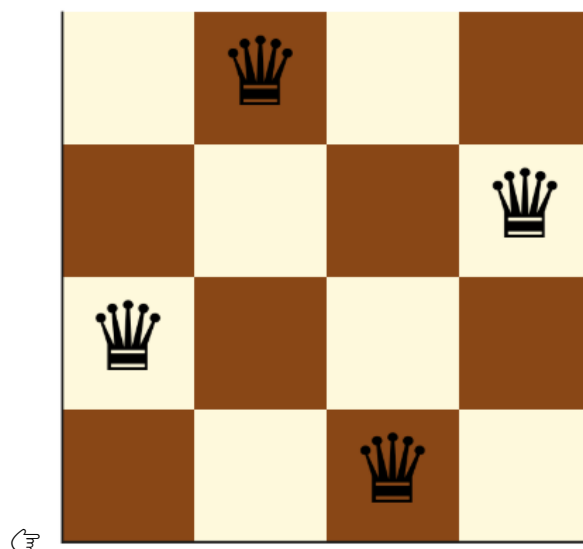
1. En la **primera fila**, la reina está en la **columna 2**.
2. En la **segunda fila**, la reina está en la **columna 4**.
3. En la **tercera fila**, la reina está en la **columna 1**.
4. En la **cuarta fila**, la reina está en la **columna 3**.

¿Por qué es válido?

Imagina que son reinas de ajedrez reales:

- Ninguna comparte la misma fila → porque solo hay una por fila.
- Ninguna comparte la misma columna → están todas en diferentes columnas.
- Ninguna está en diagonal con otra → eso lo verifica el programa antes de colocar cada reina.

Así, ninguna puede atacar a otra, lo que cumple la regla del problema.



Aquí tienes un **tablero de ajedrez de 4x4** con una solución al problema de las N-Reinas:

- Las casillas alternan blanco y marrón como en un tablero real.
- Las reinas (♔) se colocan en posiciones donde **ninguna puede atacarse entre sí**.